

AFRILANG Stdlib

std/c/mlperceptron

Français. Bibliothèque AFRILANG 'std/c/mlperceptron' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : predict, trainStep, accuracy, hingeLoss, initWeights, trainEpoch.

```
'import "std/c/mlperceptron"' · 'use mlperceptron'
```

```
- 'predict(weights list number, features list number, bias number) →  
number' - 'trainStep(weights list number, bias number, features list  
number, label number, lr number) → list number' - 'accuracy(weights  
list number, bias number, X list number, y list number, dims number) →  
number' - 'hingeLoss(weights list number, features list number, label  
number, bias number) → num
```

English. AFRILANG library 'std/c/mlperceptron' (tier complex, category complex). Exports 6 function(s): predict, trainStep, accuracy, hingeLoss, initWeights, trainEpoch.

```
'import "std/c/mlperceptron"' · 'use mlperceptron'
```

```
- 'predict(weights list number, features list number, bias number) →  
number' - 'trainStep(weights list number, bias number, features list  
number, label number, lr number) → list number' - 'accuracy(weights  
list number, bias number, X list number, y list number, dims number) →  
number' - 'hingeLoss(weights list number, features list number, label  
number, bias number) → number' - 'initW
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/mlperceptron' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : predict, trainStep, accuracy, hingeLoss, initWeights, trainEpoch.

```
'import "std/c/mlperceptron"' · 'use mlperceptron'
```

```
- `predict(weights list number, features list number, bias number) → number`
- `trainStep(weights list number, bias number, features list number, label number, lr
  number) → list number`
- `accuracy(weights list number, bias number, X list number, y list number, dims
  number) → number`
- `hingeLoss(weights list number, features list number, label number, bias number) →
  number`
- `initWeights(size number) → list number`
- `trainEpoch(weights list number, bias number, X list number, y list number, lr
  number, dims number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlperceptron"
use mlperceptron
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

```
•
predict(weights list number, features list number, bias number) → number•
trainStep(weights list number, bias number, features list number, label number, lr
  number) → list•
accuracy(weights list number, bias number, X list number, y list number, dims number
  ) → number•
hingeLoss(weights list number, features list number, label number, bias number) →
  number•
initWeights(size number) → list•
trainEpoch(weights list number, bias number, X list number, y list number, lr number
  , dims number) → list
```

4. Exemples d'utilisation

```
import "std/c/mlperceptron"
use mlperceptron
```

```
create result = predict(/* args */)
say result
```

```
create result = trainStep(/* args */)
say result
```

```
create result = accuracy(/* args */)
say result
```

```
create result = hingeLoss(/* args */)
say result
```

```
create result = initWeights(/* args */)
say result
```

```
create result = trainEpoch(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/mlperceptron"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module mlperceptron

```
function predict(weights list number, features list number, bias number) returns
  number
end
```

```
function trainStep(weights list number, bias number, features list number, label
  number, lr number) returns list number
end
```

```
function accuracy(weights list number, bias number, X list number, y list number,
  dims number) returns number
end
```

```
function hingeLoss(weights list number, features list number, label number, bias
  number) returns number
end
```

```
function initWeights(size number) returns list number
end
```

```
function trainEpoch(weights list number, bias number, X list number, y list number,
  lr number, dims number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/mlperceptron' (tier complex, category complex). Exports 6 function(s): predict, trainStep, accuracy, hingeLoss, initWeights, trainEpoch.

'import "std/c/mlperceptron"' · 'use mlperceptron'

```
- `predict(weights list number, features list number, bias number) → number`
- `trainStep(weights list number, bias number, features list number, label number, lr
  number) → list number`
- `accuracy(weights list number, bias number, X list number, y list number, dims
  number) → number`
- `hingeLoss(weights list number, features list number, label number, bias number) →
  number`
- `initWeights(size number) → list number`
- `trainEpoch(weights list number, bias number, X list number, y list number, lr
  number, dims number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlperceptron"
use mlperceptron
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

```
•
predict(weights list number, features list number, bias number) → number•
trainStep(weights list number, bias number, features list number, label number, lr
  number) → list•
accuracy(weights list number, bias number, X list number, y list number, dims number
  ) → number•
hingeLoss(weights list number, features list number, label number, bias number) →
  number•
initWeights(size number) → list•
trainEpoch(weights list number, bias number, X list number, y list number, lr number
  , dims number) → list
```

4. Usage examples

```
import "std/c/mlperceptron"
use mlperceptron
```

```
create result = predict(/* args */)
say result
```

```
create result = trainStep(/* args */)
say result
```

```
create result = accuracy(/* args */)
say result
```

```
create result = hingeLoss(/* args */)
say result
```

```
create result = initWeights(/* args */)
say result
```

```
create result = trainEpoch(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/mlperceptron"`` at the top of your file.
- Run ``afriLang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module mlperceptron

```
function predict(weights list number, features list number, bias number) returns
  number
end
```

```
function trainStep(weights list number, bias number, features list number, label
  number, lr number) returns list number
end
```

```
function accuracy(weights list number, bias number, X list number, y list number,
  dims number) returns number
end
```

```
function hingeLoss(weights list number, features list number, label number, bias
  number) returns number
end
```

```
function initWeights(size number) returns list number
end
```

```
function trainEpoch(weights list number, bias number, X list number, y list number,
  lr number, dims number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlperceptron"`
- Vérification : `afriLang check`
- Documentation : <https://afriLang.dev/stdlib/std/c/mlperceptron/>

Source (excerpt)

```
module mlperceptron

function predict(weights list number, features list number, bias number) returns
    number
end

function trainStep(weights list number, bias number, features list number, label
    number, lr number) returns list number
end

function accuracy(weights list number, bias number, X list number, y list number,
    dims number) returns number
end

function hingeLoss(weights list number, features list number, label number, bias
    number) returns number
end

function initWeights(size number) returns list number
end

function trainEpoch(weights list number, bias number, X list number, y list number,
    lr number, dims number) returns list number
end
end
```